

	id [PK] bigint	First_name character varying (50)	Last_name character varying (50)	Student_ID integer	Email_ID character varying (254)

Figure 7: Glimpse of our newly-created model

```

]
operations = [
    migrations.CreateModel(
name='Studentinfo',
fields=[
        ('id', models.
BigAutoField(auto_created=True,
primary_key=True, serialize=False,
verbose_name='ID')),
        ('First_name', models.
CharField(max_length=50)),
        ('Last_name', models.
CharField(max_length=50)),
        ('Student_ID', models.
IntegerField()),
        ('Email_ID', models.
EmailField(max_length=254)),
    ],
    ),
]

```

For the next step, we simply use the keyword 'migrate', which results in a table called 'Studentinfo' being created in our database:

```
(venv1) C:\Users\kamil\osfydir\
osfyproject1>python manage.py migrate
```

The output of the above command shows around 15 items being checked as 'OK'. If you take a look at the 'Tables' tab in the 'osfyproject1' database, you will find a 'firstapp_studentinfo' table, which is shown in Figure 7.

The 'id' field has been assigned as the primary key of the table by default. You can set another field as the primary key by including 'primary_key=True' as one of the parameters of the field within the model definition in 'models.py'. Now let's jump into performing CRUD operations for this model.

Create a new file within 'firstapp' called 'serializers.py' and write the following:

```

from rest_framework import serializers
from .models import Studentinfo

class Studentserializer(serializers.
ModelSerializer):
    class Meta:
        model=Studentinfo
        fields='__all__'

```

A serializer is defined on the Web as follows: "Serializers allow complex data such as query sets and model instances to be converted to native Python datatypes that can then be easily rendered into JSON." Simply put, it converts the object(s) entered into JSON format.

Let's take an example. Say, we have a simple logic for entering new data into the database in our 'views.py', which does not include checks for any errors. Now, if we were to make an entry into our table in the format below:

```

# create a student object
>>> student = Studentinfo(First_
name='Kamil', Last_name='Khan',
Student_ID=085, Email_ID='kamil123@
gmail.com')

# serialize the data
>>> serializer =
Studentserializer(student)

# print serialized data
>>> serializer.data

```

...the output of 'serializer.data' would have been as follows:

```
{'First_name': 'Kamil', 'Last_name':
'Khan', 'Student_ID': 085, 'Email_ID':
'kamilpaigah@gmail.com'}
```

The aim of covering the above code was to provide a practical explanation of the definition found on the Web, but since we will be using Insomnia here onwards, we will not be required to enter data in the format we just observed. We will be entering data in JSON format itself. In simple terms, a serializer creates a serializer class, which maps the fields of the model it is associated with. You may ask, if that's the case, what's the difference between a model and a serializer? From what I have observed, a serializer has the capability to check the validity of the data entered, such as checking if the value entered for field(s) is of the type defined in the model; if not, it will show errors. Now let's move on to 'views.py', wherein we will cover the logic for 'Create' and 'Read'.

```

from firstapp.models import Studentinfo
from .serializers import
Studentserializer
from rest_framework.response import
Response
from rest_framework import status
# Create your views here.
---(5)

class Student_post_get_view (APIView):
    def post(self,request):
        serializer=Studentserializer(da
ta=request.data)
        if serializer.is_valid():
            ---(10)
            serializer.save()
            return Response(serializer.

```